Project Report

“Designing a Pipelined RISC-V like Processor”

Under the Course,

“High Performance Computing Architecture”

Department of Electronics and Communication Engineering

National Institute of Technology Surathkal, Mangaluru – 575025, Karnataka

Course Instructor :-

M. S. Bhat (Professor, NITK)

Project by :-

Rishabh Barwe (252SP025)

Konatham Praveen Raj (252SP014)

1. Introduction and Architectural Plan

This report outlines the hardware design and Instruction Set Architecture (ISA) for a custom 32-bit pipelined processor based on the RISC-V RV32I base integer specification. The architecture implements a classical 5-stage pipeline: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB).

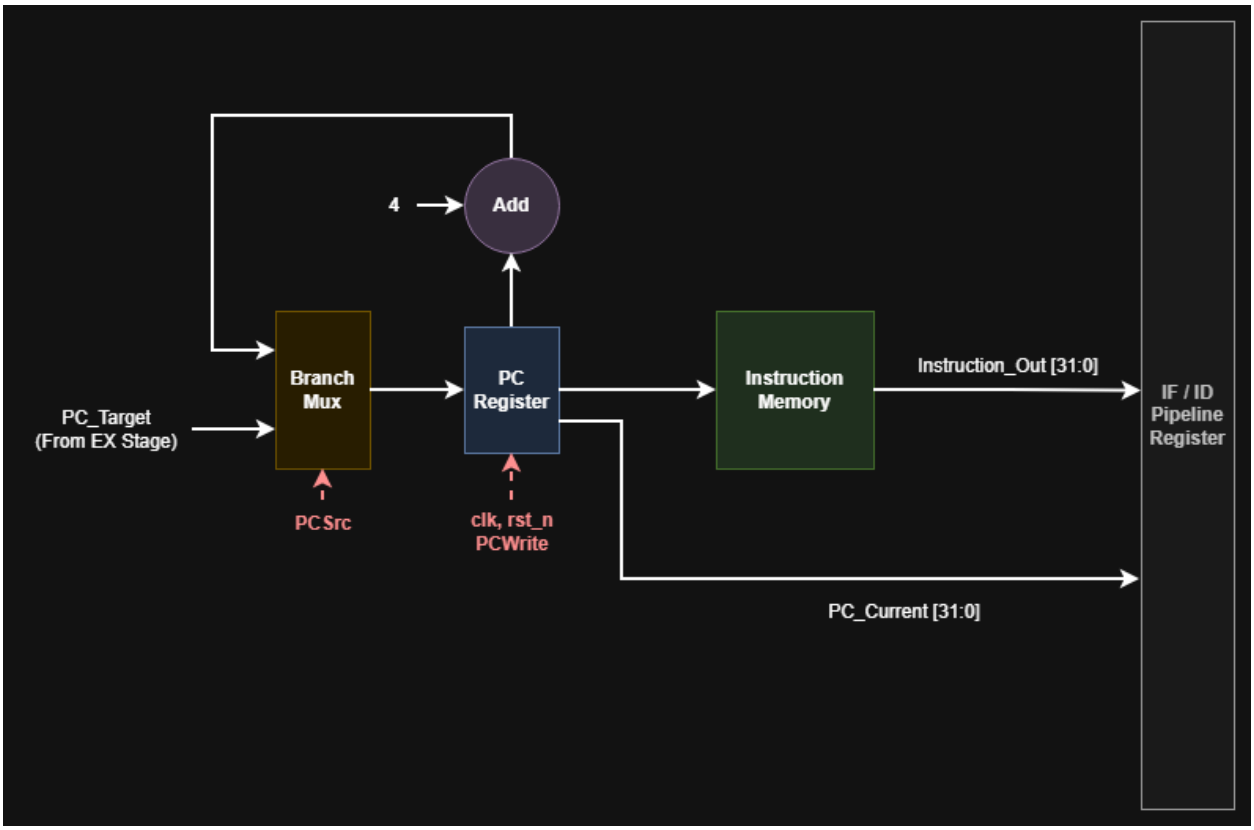
To ensure high performance and minimize pipeline stalling, our this design incorporates a **Data Forwarding Unit**. In a standard pipeline, Read-After-Write (RAW) hazards occur when an instruction requires a register value that is currently being computed by a previous, still-executing instruction. Instead of stalling the pipeline until the data reaches the Write-Back stage, our Forwarding Unit detects if the destination register (rd) of an instruction in the EX/MEM or MEM/WB stages matches the source registers (rs1, rs2) of the instruction in the ID/EX stage. If a match is detected, the unit multiplexes the newly computed data directly into the ALU inputs, ensuring seamless execution.

2. Hardware Blocks and Signals

The datapath is divided into five distinct stages separated by pipeline registers. Below are the specific input, output, and control signals for the primary hardware blocks in each stage.

2.1 Instruction Fetch (IF) Stage

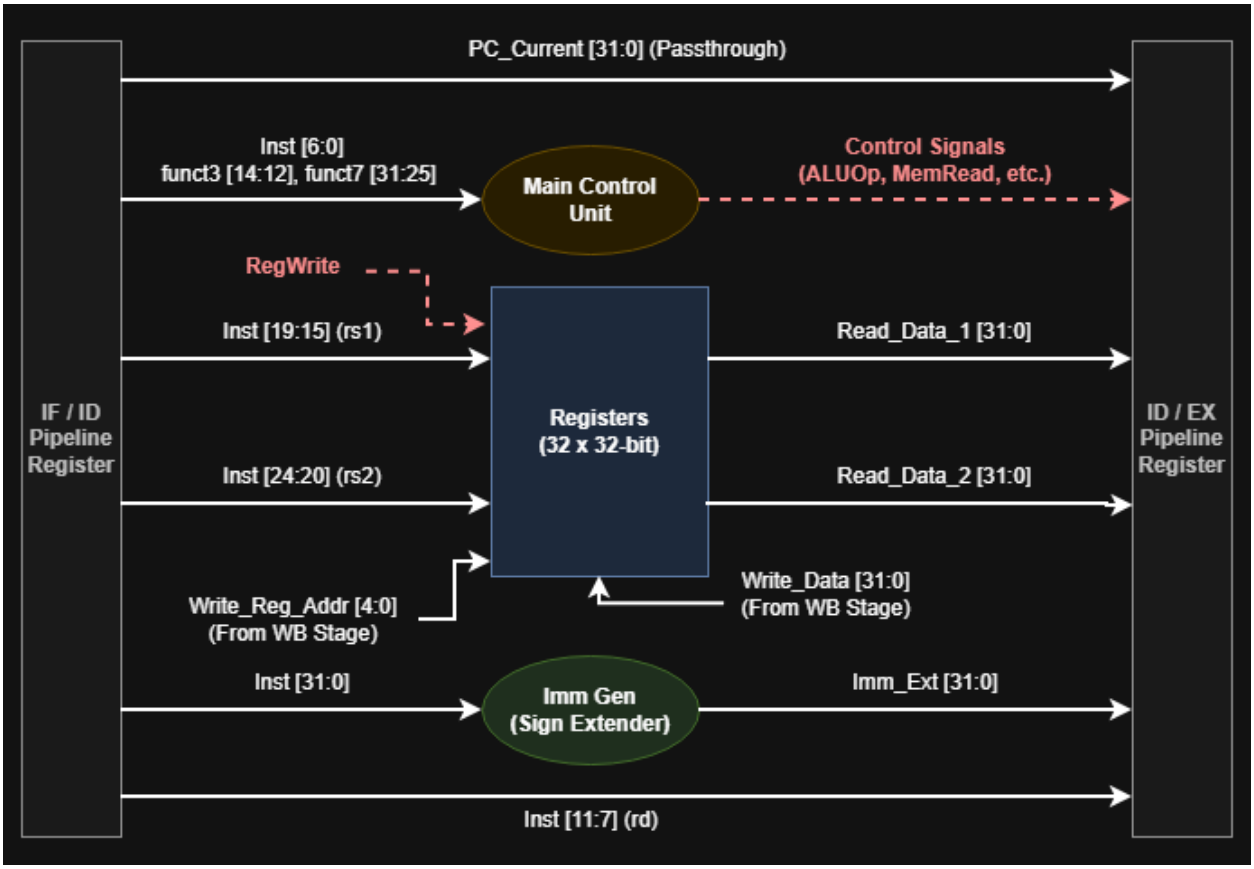
- **Primary Block:** Program Counter (PC) & Instruction Memory.
- **Input Signals:** clk, reset (active low), PC_Target (32-bit, from EX stage), PCSrc (1-bit control for branch multiplexing).
- **Output Signals:** Instruction_Out (32-bit to ID), PC_Current (32-bit to ID).
- **Control Signals:** PCWrite (to stall IF during Load-Use hazards).



Block diagram of IF stage

2.2 Instruction Decode (ID) Stage

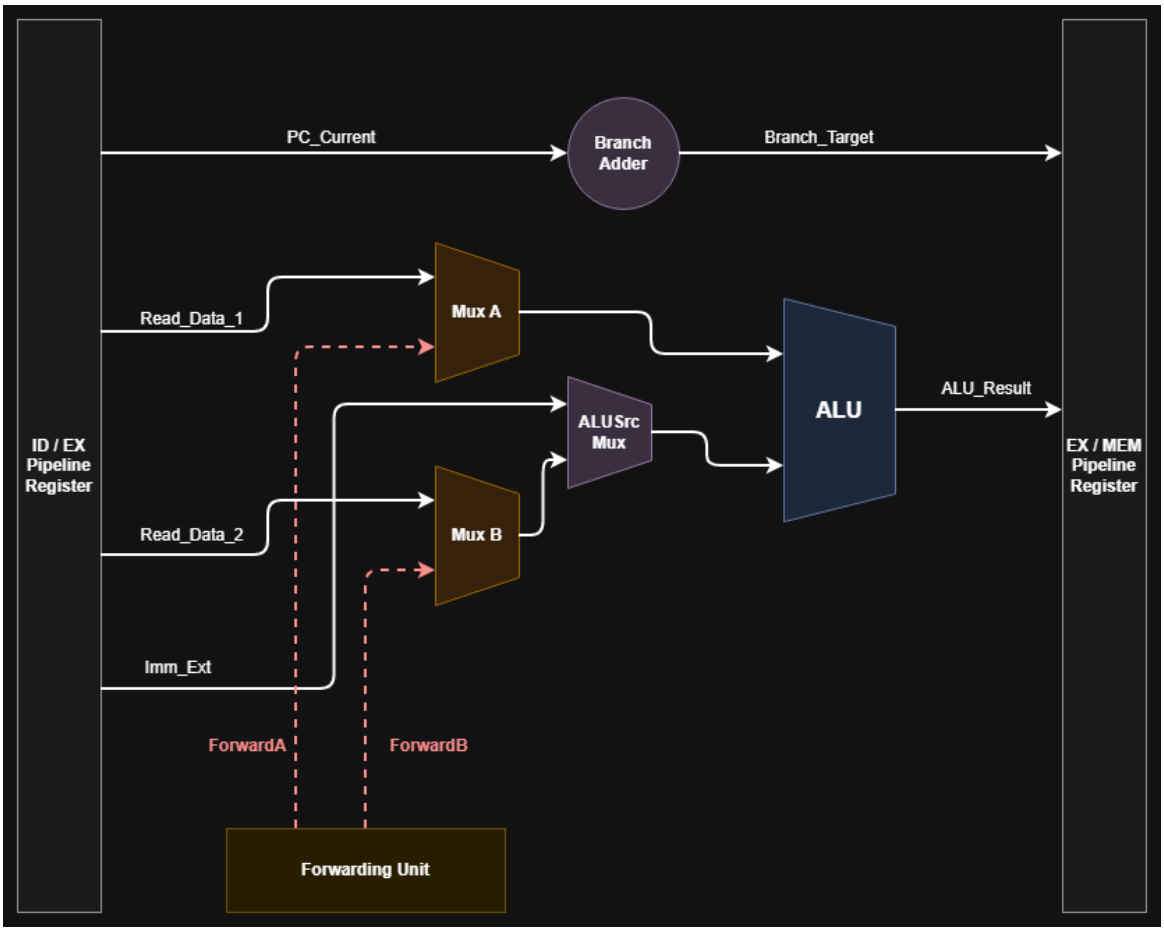
- **Primary Block:** 32x32-bit Register File & Control Unit.
- **Input Signals:** Instruction_In (32-bit), Write_Data (32-bit, from WB), Write_Reg_Addr (5-bit).
- **Output Signals:** Read_Data_1 (32-bit), Read_Data_2 (32-bit), Imm_Ext (32-bit sign-extended immediate).
- **Control Signals:** RegWrite (1-bit input), ALUOp (output), MemRead (output), MemWrite (output), Branch (output).



Block diagram of ID stage

2.3 Execute (EX) Stage

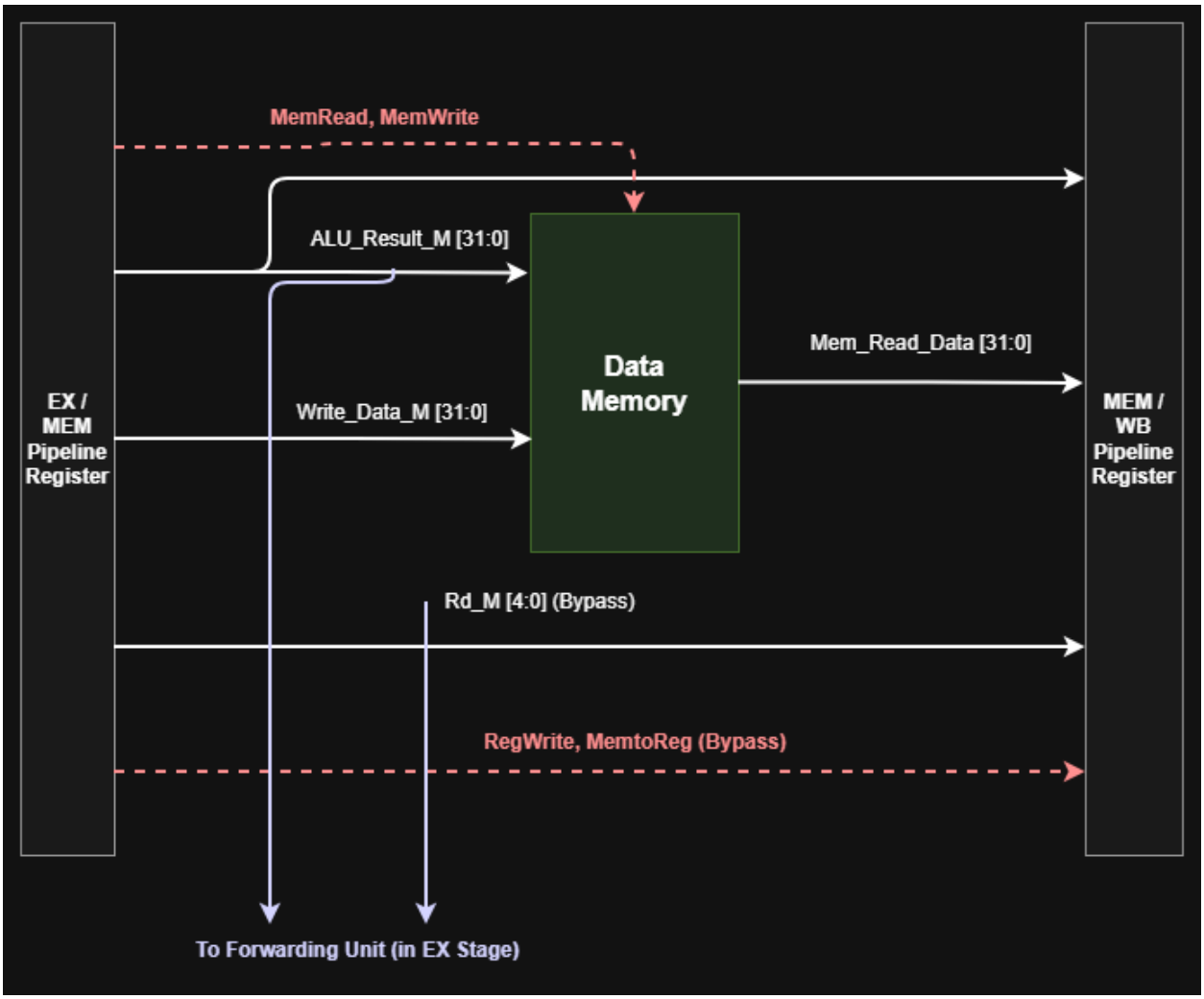
- **Primary Block:** Arithmetic Logic Unit (ALU) & Forwarding Unit.
- **Input Signals:** Op_A and Op_B (32-bit, selected via forwarding muxes), Imm_Ext (32-bit).
- **Output Signals:** ALU_Result (32-bit), Zero_Flag (1-bit), Branch_Target_Addr (32-bit).
- **Control Signals:** ALU_Control (4-bit input), ForwardA, ForwardB (2-bit outputs from Forwarding Unit to control MUXes).



Block diagram of EX stage (Branch part in EX design is incomplete and will be added later with branch predictor)

2.4 Memory Access (MEM) Stage

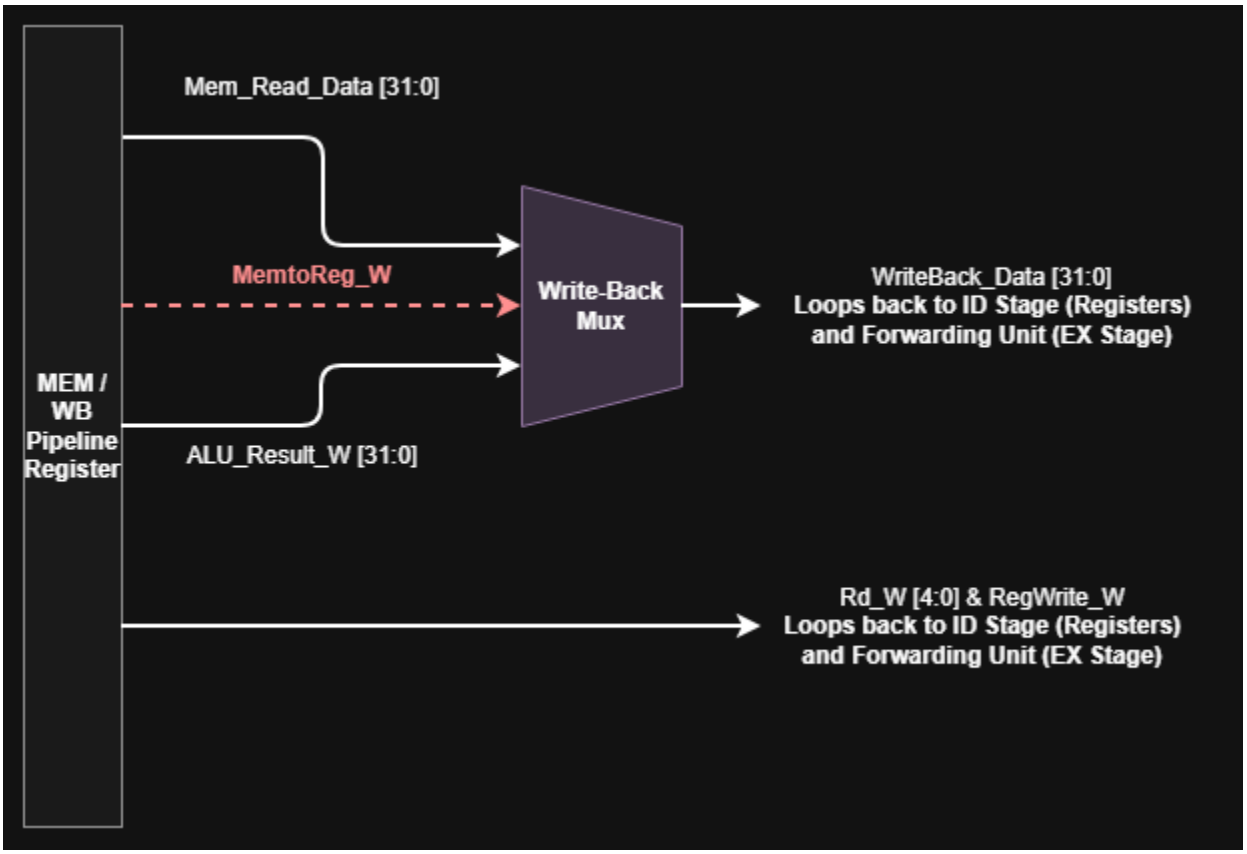
- **Primary Block:** Data Memory (DMEM).
- **Input Signals:** ALU_Result (acts as 32-bit Address), Write_Data (32-bit, forwarded from EX/MEM register).
- **Output Signals:** Read_Data_Out (32-bit).
- **Control Signals:** MemRead_Enable, MemWrite_Enable (1-bit inputs).



Block diagram of mem stage

2.5 Write-Back (WB) Stage

- **Primary Block:** Write-Back Multiplexer.
- **Input Signals:** Read_Data_Out (32-bit, from Memory), ALU_Result (32-bit, from EX).
- **Output Signals:** Result_WB (32-bit bus routed back to the ID Register File).
- **Control Signals:** MemtoReg (1-bit input to select between Memory Data or ALU Data).



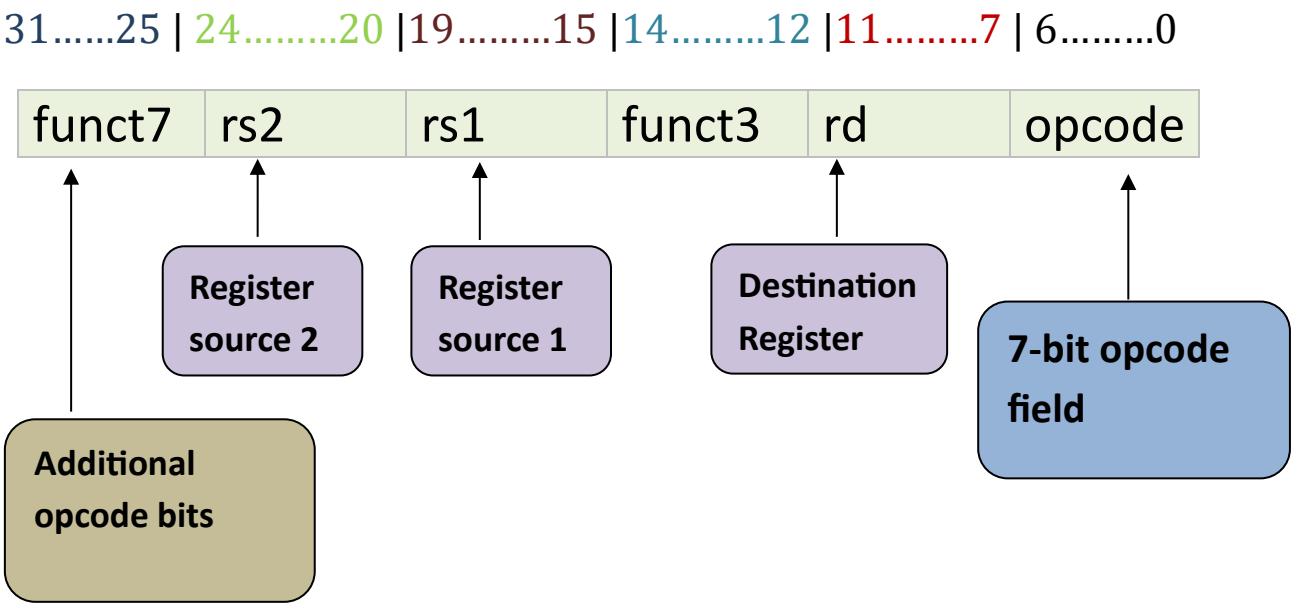
Block diagram of WB stage

3. Instruction Set Architecture (ISA) Specification

The processor implements a tailored subset of the RV32I base instruction set. The instructions utilize standard 32-bit formats. Below is the structured breakdown of the binary configurations, assembly formats, and Register Transfer Level (RTL) meanings.

3.1 R-Type Instructions (Register-Register)

- **Structure:** funct7[31:25] | rs2[24:20] | rs1[19:15] | funct3[14:12] | rd[11:7] | opcode[6:0]



Instruction	Assembly Syntax	RTL Meaning
ADD	**add rd, rs1, rs2**	Reg[rd] <- Reg[rs1] + Reg[rs2]
SUB	**sub rd, rs1, rs2**	Reg[rd] <- Reg[rs1] - Reg[rs2]
AND	**and rd, rs1, rs2**	Reg[rd] <- Reg[rs1] & Reg[rs2]
OR	**or rd, rs1, rs2**	Reg[rd] <- Reg[rs1] | Reg[rs2]

3.2 I-Type Instructions (Immediate & Load)

- **Structure** : imm[31:20] | rs1[19:15] | funct3[14:12] | rd[11:7] | opcode[6:0]

31……20 | 19……15 |14……12 |11……7 | 6……0

imm	rs1	funct3	rd	opcode

Instruction	Assembly Syntax	RTL Meaning
ADDI	**addi rd, rs1, imm**	Reg[rd] <- Reg[rs1] + SignExt(imm)
LW	**lw rd, imm(rs1)**	Reg[rd] <- Mem[Reg[rs1] + SignExt(imm)]

3.3 S-Type Instructions (Store)

- **Structure**: imm[31:25] | rs2[24:20] | rs1[19:15] | funct3[14:12] | imm[11:7] | opcode[6:0]

31.....25 | 24.....20 |19.....15 |14.....12 |11.....7 | 6.....0

imm	rs2	rs1	funct3	imm	opcode

Instruction	Assembly Syntax	RTL Meaning
SW	**sw rs2, imm(rs1)**	Mem[Reg[rs1] + SignExt(imm)] <- Reg[rs2]$

3.4 B-Type Instructions (Branch)

- **Structure:** imm[12|10:5] | rs2[24:20] | rs1[19:15] | funct3[14:12] | imm[4:1|11] | opcode[6:0]

31……25 | 24……20 |19……15 |14……12 |11……7 | 6……0

imm	rs2	rs1	funct3	imm	opcode

Instruction	Assembly Syntax	RTL Meaning
BEQ	**beq rs1, rs2, imm**	If(Reg[rs1] == Reg[rs2]) PC <- PC + SignExt(imm) * 2

3.5 U-Type & J-Type Instructions (Upper Immediate & Jump)

- **U-Type Structure:** imm[31:12] | rd[11:7] | opcode[6:0]
- **J-Type Structure:** imm[20|10:1|11|19:12] | rd[11:7] | opcode[6:0]

31……..12 |11……..17 |6……..0

imm	rd	opcode

U-type

31……12 |11…..17 |6……..0

imm[20|10:1|11|19:12]	rd	opcode

UJ-type

Instruction	**Type**	**Assembly Syntax**	**RTL Meaning**
LUI	U	**lui rd, imm**	Reg[rd] <- imm << 12
JAL	J	**jal rd, imm**	Reg[rd] <- PC + 4; [cite_start]PC <- PC + SignExt(imm) * 2

4. Future Development and Optimization

As the structural Verilog coding phase begins for Cadence synthesis, further optimizations are being evaluated to maximize the processor's efficiency. While the Data Forwarding Unit will resolve standard data hazards, control hazards resulting from branch instructions still risk introducing pipeline flushes. To mitigate this and further improve the Instructions Per Cycle (IPC) throughput, we plan to design and integrate a dedicated hardware branch predictor during the final development phase.